

Week 35: Ridge and Lasso Regression

Gauss–Markov, and why a little bias is a good investment

With pauses for questions

Morten Hjorth-Jensen

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway

Plan for week 35

Monday lecture (2×45 min):

- ▶ Reminder from week 34, and a first look at the results of *exercise 3* of this week's set: training vs test error
- ▶ Measures of quality (Section 3.5): MSE, R^2 and where it comes from, MAE
- ▶ The Gauss–Markov theorem – what it promises and what it does not
- ▶ Ridge regression: deriving the SVD form, Eqs. (3.46) and (3.47), and reading the shrinkage factor, Eq. (3.48); bias, variance, Hoerl–Kennard
- ▶ The Lasso: soft thresholding – Eqs. (3.60), (3.61) and (3.63) plotted and discussed – sparsity and the geometry of the 1-norm
- ▶ The three estimators on one problem (Section 3.10)

Tuesday session (2×45 min, flipped): your questions, the exercise-3 figure revisited, and Section 3.13 – scaling, no-scaling and the intercept for OLS, Ridge and Lasso – as hands-on cases. Bring your laptop.

How the slides work

Every few slides there is a *Pause and think* frame. Talk to your neighbour for a minute; then we discuss. Reading: Chapter 3, Sections 3.5, 3.7–3.10 and 3.13 of the book.

Reminder from week 34

The linear model $y = X\theta + \varepsilon$, with the mean squared error as cost function, gave the normal equations and the optimal model parameters

$$\hat{\theta}_{\text{OLS}} = (X^T X)^{-1} X^T y.$$

Through the SVD $X = U\Sigma V^T$ – the way we actually compute it – the parameters and the model $\tilde{y} = X\hat{\theta}$ read

$$\hat{\theta}_{\text{OLS}} = \sum_{i=0}^{p-1} \frac{u_i^T y}{\sigma_i} v_i, \quad \tilde{y}_{\text{OLS}} = X\hat{\theta}_{\text{OLS}} = UU^T y = \sum_{i=0}^{p-1} u_i u_i^T y :$$

the fit expresses the data in the orthonormal basis U of the column space and keeps every coordinate untouched – the projection picture of last week. Keep both expressions in mind: today's Ridge regression will change them in exactly one place.

This week's exercise 3: does a better fit predict better?

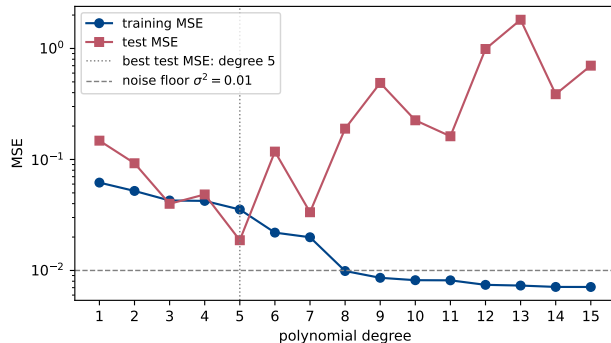
Exercise 3 of the weekly set asks you to reproduce Figure 2.11 of Hastie et al.: fit polynomials of increasing degree and compare the error on the data used for the fit with the error on data held out.

```
rng = np.random.default_rng(2026)
n = 100
x = np.linspace(-3, 3, n)
y = np.exp(-x**2) + 1.5*np.exp(-(x - 2)**2) + rng.normal(0, 0.1, n)
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3,
                                                    random_state=2026)

for d in range(1, 16):
    Xtr = np.vander(x_train, d + 1, increasing=True)
    Xte = np.vander(x_test, d + 1, increasing=True)
    theta = np.linalg.lstsq(Xtr, y_train, rcond=None)[0] # SVD based
    mse_train.append(MSE(y_train, Xtr @ theta))
    mse_test.append(MSE(y_test, Xte @ theta))
```

Same data-generating function, same noise level $\sigma = 0.1$ everywhere; the only thing that changes is the *complexity* of the model, the polynomial degree. The complete code is in this week's notebook, `week35tuesday.ipynb`.

Exercise 3: the result



- ▶ The training MSE falls *monotonically* with the degree – it must.
- ▶ The test MSE has a minimum (here at degree 5) and then grows by orders of magnitude: the model fits the noise of the training set.
- ▶ The dashed line is the noise floor $\sigma^2 = 0.01$: no model can honestly beat it.

We return to this figure in **Tuesday's flipped session**, and – with the bias–variance decomposition – next week.

Pause and think 1: reading the U-curve

Question

Three questions about the figure: (a) why can the training MSE never increase when the degree grows? (b) the training MSE dips *below* the noise floor σ^2 at high degree – is the model better than the noise allows? (c) if we doubled the number of data points, which of the two curves would move, and in which direction?

Pause and think 1: reading the U-curve

Question

Three questions about the figure: (a) why can the training MSE never increase when the degree grows? (b) the training MSE dips *below* the noise floor σ^2 at high degree – is the model better than the noise allows? (c) if we doubled the number of data points, which of the two curves would move, and in which direction?

Answer

(a) A degree- d polynomial is a special case of degree $d+1$ (set the new coefficient to zero), so the minimal training RSS can only fall. (b) No – it is fitting the particular noise realisation of the training points, “memorising” fluctuations. Training error below σ^2 is a symptom of overfitting, not of skill. (c) Mostly the test curve: with more data the high-degree fits are better constrained, the upturn moves to higher degree and becomes milder; the training curve rises slightly towards σ^2 . Try it in the notebook tomorrow.

Measures of quality (Section 3.5)

Before comparing models we must agree on how to score them.

$$\text{MSE}(y, \tilde{y}) = \frac{1}{n} \sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2, \quad R^2(y, \tilde{y}) = 1 - \frac{\sum_i (y_i - \tilde{y}_i)^2}{\sum_i (y_i - \bar{y})^2}, \quad \bar{y} = \frac{1}{n} \sum_i y_i.$$

- ▶ The MSE is what we minimise – but it carries the squared units of the target, so its absolute value says little on its own.
- ▶ R^2 is dimensionless: the model against the best *constant* prediction. $R^2 = 1$ perfect, $R^2 = 0$ no better than the mean, negative values – entirely possible on test data – worse than the mean.
- ▶ The mean absolute error $\text{MAE} = \frac{1}{n} \sum_i |y_i - \tilde{y}_i|$ is the 1-norm counterpart, far less sensitive to outliers (squaring gives a 10σ point $100\times$ the weight; the absolute value $10\times$).
- ▶ The relative error $|y_i - \tilde{y}_i|/|y_i|$ is natural when targets span decades, dangerous near $y_i = 0$.

Where R^2 comes from – and why the training R^2 lies

For a least-squares fit *with an intercept* the definition is forced by geometry. $\mathbf{H}$ projects, so $\tilde{\mathbf{y}} \perp \mathbf{e}$; and the constant column 1 in $\mathbf{X}$ makes the residuals sum to zero ($\mathbf{1}^T \mathbf{e} = 0$, so $\bar{\tilde{\mathbf{y}}} = \bar{\mathbf{y}}$). Pythagoras on the centred vectors then gives the *analysis-of-variance identity*

$$\underbrace{\sum_i (y_i - \bar{y})^2}_{\text{TSS}} = \underbrace{\sum_i (\tilde{y}_i - \bar{y})^2}_{\text{ESS}} + \underbrace{\sum_i (y_i - \tilde{y}_i)^2}_{\text{RSS}},$$

$$R^2 = 1 - \frac{\text{RSS}}{\text{TSS}} = \frac{\text{ESS}}{\text{TSS}} = \cos^2 \angle (\mathbf{y} - \bar{y}\mathbf{1}, \tilde{\mathbf{y}} - \bar{y}\mathbf{1}) \in [0, 1].$$

Off the training set the residuals neither sum to zero nor stay orthogonal to the fit: the identity breaks and R^2 can go negative.

- ▶ Adding *any* column – even pure noise – can only lower the training RSS, so the training R^2 only rises; at $p = n$ it is 1 regardless.
- ▶ The adjusted $\bar{R}^2 = 1 - (1 - R^2) \frac{n-1}{n-p}$ penalises useless columns – a palliative, not a cure.

The standing rule of this course

All three measures mean something only on data held out from the fit. Every number we report is a test quantity unless said otherwise – exercise 3 above is the picture to keep in mind.

Pause and think 2: a negative R^2

Question

A fellow student reports $R^2 = -0.3$ on the test set and concludes the code must be wrong, since “ R^2 is a squared cosine and lies in $[0, 1]$ ”. What do you answer? And what is the simplest model that achieves $R^2 = 0$ on any test set – so what does $R^2 < 0$ actually tell us?

Pause and think 2: a negative R^2

Question

A fellow student reports $R^2 = -0.3$ on the test set and concludes the code must be wrong, since “ R^2 is a squared cosine and lies in $[0, 1]$ ”. What do you answer? And what is the simplest model that achieves $R^2 = 0$ on any test set – so what does $R^2 < 0$ actually tell us?

Answer

The $\cos^2$ identity is a *training-set* statement: it needs the residuals to sum to zero and to be orthogonal to the fit, which the normal equations guarantee only on the data used in the fit. On test data nothing protects the sign. Predicting the constant $\bar{y}$ (of the training targets) gives $R^2 \approx 0$; a negative value therefore means the fitted model predicts *worse than the mean of the data* – perfectly possible for an overfitted model, and exactly what the right-hand side of the exercise-3 figure shows. The code may be fine; the model is not.

The Gauss–Markov theorem

Among all estimators that are *linear* in y and *unbiased*, OLS has the smallest variance. Only zero mean, constant variance and uncorrelated noise are needed – *not* Gaussianity.

Theorem (Gauss–Markov)

Let $y = X\theta + \varepsilon$ as above, X of full column rank. If $\tilde{\theta} = Cy$ is linear and unbiased for every θ , then $\text{Var}(\tilde{\theta}) - \text{Var}(\hat{\theta}_{\text{OLS}})$ is positive semi-definite.

Proof. Unbiasedness for every θ forces $CX = I$. Write $C = (X^T X)^{-1} X^T + D$; then $DX = 0$, and

$$\text{Var}(\tilde{\theta}) = \sigma^2 C C^T = \sigma^2 (X^T X)^{-1} + \sigma^2 D D^T,$$

the cross terms vanishing because $DX = 0$. The second term is a Gram matrix, positive semi-definite, zero only for $D = 0$. □

Every linear unbiased competitor is OLS *plus* a piece Dy that averages to zero and only adds variance.

Pause and think 3: the fine print of Gauss–Markov

Question

“OLS is the best linear unbiased estimator, so nothing can beat it.” Find the two loopholes in that sentence – which restrictions does the theorem place on the competition, and why might we want to violate each of them?

Pause and think 3: the fine print of Gauss–Markov

Question

“OLS is the best linear unbiased estimator, so nothing can beat it.” Find the two loopholes in that sentence – which restrictions does the theorem place on the competition, and why might we want to violate each of them?

Answer

Linear and *unbiased*. Nonlinear estimators are excluded (and for heavy-tailed noise beat OLS easily – recall the Laplace/MAE discussion of week 34). More importantly, *biased* estimators were never in the competition, and the mean squared error

$$\mathbb{E}[\|\hat{\boldsymbol{\theta}} - \boldsymbol{\theta}\|^2] = \text{bias}^2 + \text{variance}$$

counts bias and variance equally. If a little bias buys a lot of variance, the trade is profitable. The rest of today constructs exactly such estimators.

Ridge regression: the definition

Add a penalty on the squared length of the parameter vector,

$$\min_{\boldsymbol{\theta} \in \mathbb{R}^p} \left\{ \frac{1}{n} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2 + \lambda \|\boldsymbol{\theta}\|_2^2 \right\}, \quad \lambda \geq 0,$$

with the closed-form solution (conventional scaling of λ)

$$\hat{\boldsymbol{\theta}}_{\text{Ridge}} = \left(\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I} \right)^{-1} \mathbf{X}^T \mathbf{y}.$$

- ▶ OLS with a modified diagonal: whatever the rank of $\mathbf{X}$, all eigenvalues of $\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}$ are at least λ , so the inverse exists for every $\lambda > 0$ – *even when $p > n$* .
- ▶ Equivalent *constrained* form: minimise the squared error subject to $\|\boldsymbol{\theta}\|_2^2 \leq t$, a ball in parameter space, with a one-to-one decreasing map between t and λ (Lagrange/KKT). Remember the ball – the Lasso will replace it by a diamond.
- ▶ Two limits: $\lambda \rightarrow 0$ recovers OLS; $\lambda \rightarrow \infty$ drives $\hat{\boldsymbol{\theta}} \rightarrow 0$.

Deriving Eq. (3.46): the Ridge estimator in the singular basis

Insert the (economy-size) SVD $X = U\Sigma V^T$, with $U^T U = V^T V = VV^T = I_p$ and $\Sigma = \text{diag}(\sigma_0, \dots, \sigma_{p-1})$, into the Ridge solution. Step by step:

$$X^T X = V \Sigma U^T U \Sigma V^T = V \Sigma^2 V^T, \quad X^T X + \lambda I = V (\Sigma^2 + \lambda I) V^T,$$

using $I = VV^T$ in the second step – the penalty is diagonal *in every basis*. Orthogonal matrices invert by transposition, diagonal matrices entry by entry:

$$(X^T X + \lambda I)^{-1} = V (\Sigma^2 + \lambda I)^{-1} V^T, \quad X^T y = V \Sigma U^T y.$$

Multiplying the two and reading off components ($V^T V = I$),

$$\hat{\theta}_{\text{Ridge}} = V (\Sigma^2 + \lambda I)^{-1} \Sigma U^T y = \sum_{i=0}^{p-1} \frac{\sigma_i}{\sigma_i^2 + \lambda} (u_i^T y) v_i \quad (\text{Eq. 3.46}).$$

Setting $\lambda = 0$ recovers the OLS expansion with coefficients $1/\sigma_i$: *the only change Ridge makes is $1/\sigma_i \rightarrow \sigma_i/(\sigma_i^2 + \lambda)$.*

Deriving Eq. (3.47), and reading Eq. (3.48)

The fitted values follow by one more multiplication:

$$\tilde{y}_{\text{Ridge}} = X\hat{\theta}_{\text{Ridge}} = U \underbrace{\Sigma V^T V}_I (\Sigma^2 + \lambda I)^{-1} \Sigma U^T y = \sum_{i=0}^{p-1} u_i u_i^T \frac{\sigma_i^2}{\sigma_i^2 + \lambda} y \quad (\text{Eq. 3.47}),$$

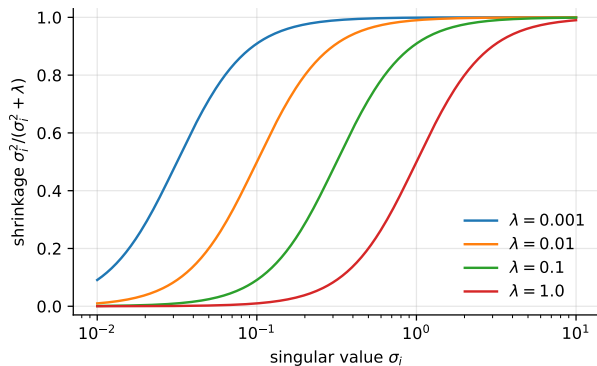
while for OLS the same expression holds with every factor replaced by one: $\tilde{y}_{\text{OLS}} = U U^T y$, the hat matrix in the singular basis. Since $\lambda \geq 0$,

$$\frac{\sigma_i^2}{\sigma_i^2 + \lambda} \leq 1 \quad (\text{Eq. 3.48}).$$

What Eq. (3.48) says

Ridge expresses y in the orthonormal basis U and *shrinks* each coordinate. The singular values are ordered descendingly, so the shrinkage is mild for the leading directions and severe for the trailing ones – precisely the directions whose OLS variance σ^2/σ_i^2 blew up last week. The effective number of parameters, $\text{df}(\lambda) = \sum_i \sigma_i^2/(\sigma_i^2 + \lambda)$, falls smoothly from p at $\lambda = 0$ towards zero: λ is a *continuous* complexity dial, where the polynomial degree was a discrete one.

The shrinkage factor of Eq. (3.48) is a smooth switch



- ▶ Directions with $\sigma_i^2 \gg \lambda$ pass through untouched; directions with $\sigma_i^2 \ll \lambda$ are suppressed almost entirely.
- ▶ The transition is centred on $\sigma_i = \sqrt{\lambda}$; increasing λ slides the switch to the right, discarding more of the spectrum.
- ▶ Truncated-SVD regression replaces the smooth curve by a hard step – that is the *only* difference between the two methods.

Pause and think 4: the orthonormal special case

Question

Let the design be orthonormal, $X^T X = I$ (all $\sigma_i = 1$). Write down $\hat{\theta}_{\text{Ridge}}$ in terms of $\hat{\theta}_{\text{OLS}}$. Does Ridge ever set a coefficient *exactly* to zero?

Pause and think 4: the orthonormal special case

Question

Let the design be orthonormal, $X^T X = I$ (all $\sigma_i = 1$). Write down $\hat{\theta}_{\text{Ridge}}$ in terms of $\hat{\theta}_{\text{OLS}}$. Does Ridge ever set a coefficient *exactly* to zero?

Answer

$\hat{\theta}_{\text{Ridge}} = (I + \lambda I)^{-1} X^T y = \hat{\theta}_{\text{OLS}} / (1 + \lambda)$: every coefficient is scaled by the *same* factor $1/(1 + \lambda)$, reaching zero only in the limit $\lambda \rightarrow \infty$. Ridge shrinks proportionally; it never selects. Keep this result – it is Eq. (3.61), and later in this lecture it will be plotted next to the Lasso's soft thresholding, Eq. (3.63).

Ridge is biased – and has uniformly smaller variance

Taking expectations and variances of the closed-form solution,

$$\mathbb{E} \left[\hat{\boldsymbol{\theta}}_{\text{Ridge}} \right] = \left(\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I} \right)^{-1} \left(\mathbf{X}^T \mathbf{X} \right) \boldsymbol{\theta} \neq \boldsymbol{\theta} \quad (\lambda > 0),$$

$$\text{Var} \left(\hat{\boldsymbol{\theta}}_{\text{Ridge}} \right) = \sigma^2 \left[\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I} \right]^{-1} \mathbf{X}^T \mathbf{X} \left\{ \left[\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I} \right]^{-1} \right\}^T,$$

and subtracting,

$$\text{Var}(\hat{\boldsymbol{\theta}}_{\text{OLS}}) - \text{Var}(\hat{\boldsymbol{\theta}}_{\text{Ridge}}) = \sigma^2 \left[\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I} \right]^{-1} \left[2\lambda \mathbf{I} + \lambda^2 (\mathbf{X}^T \mathbf{X})^{-1} \right] \left\{ \left[\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I} \right]^{-1} \right\}^T,$$

which is positive semi-definite for every $\lambda > 0$: the Ridge variance is *always* smaller.

The trade in one sentence

Increasing λ increases the squared bias and decreases the variance, both monotonically; the mean squared error is their sum. Gauss–Markov is not contradicted – Ridge is biased, so it was never in that competition.

A useful penalty always exists (Hoerl–Kennard)

Work in the singular basis, $b_i = (V^T \theta)_i$. The total mean squared error of the Ridge estimator is

$$M(\lambda) = \mathbb{E} \left\| \hat{\theta}_{\text{Ridge}} - \theta \right\|_2^2 = \sum_{i=0}^{p-1} \frac{\overbrace{\lambda^2 b_i^2}^{\text{squared bias}} + \overbrace{\sigma^2 \sigma_i^2}^{\text{variance}}}{(\sigma_i^2 + \lambda)^2}.$$

Theorem (Hoerl–Kennard)

For full column rank and $\sigma^2 > 0$,

$$M'(0) = -2\sigma^2 \sum_{i=0}^{p-1} \frac{1}{\sigma_i^4} < 0,$$

so some $\lambda > 0$ gives $M(\lambda) < M(0)$: a Ridge estimator beats OLS in mean squared error *whatever the true θ* .

Two morals. The bias enters as λ^2 – zero slope at the origin – while the variance falls linearly: *a little bias is free to second order*. And the slope is dominated by $1/\sigma_{\min}^4$: the worse the collinearity, the more Ridge has to offer.

Pause and think 5: an existence theorem, not a recipe

Question

Hoerl–Kennard guarantees a good λ exists. Look at the formula for $M(\lambda)$: which quantities in it do we *not* know in practice? So how do we actually choose λ ? And why does the theorem predict Ridge helps most exactly when the design is badly conditioned?

Pause and think 5: an existence theorem, not a recipe

Question

Hoerl–Kennard guarantees a good λ exists. Look at the formula for $M(\lambda)$: which quantities in it do we *not* know in practice? So how do we actually choose λ ? And why does the theorem predict Ridge helps most exactly when the design is badly conditioned?

Answer

$M(\lambda)$ depends on the true coefficients b_i and the true noise σ^2 – unknown, otherwise we would not be fitting. In practice λ is chosen by cross-validation on a grid (next week; the exercises already let you try it). The slope $M'(0) = -2\sigma^2 \sum_i \sigma_i^{-4}$ is dominated by the smallest singular value, i.e. by the direction where OLS variance $\sigma^2/\sigma_{\min}^2$ explodes – that is where cheap variance reduction is available, and week 34's collinearity cliffhanger is resolved.

Second lecture. A preview: scaling and the intercept (Section 3.13)

Before we meet the Lasso, one practical warning that applies to *every* penalised method – treated in full, hands-on, in tomorrow's flipped session.

- ▶ The penalties $\|\boldsymbol{\theta}\|_2^2$ and $\|\boldsymbol{\theta}\|_1$ treat all coefficients alike, but a coefficient's size depends on the *units* of its feature: for Ridge and Lasso a change of units is a change of model. (OLS is exempt – its predictions are equivariant under column rescaling.) Remedy: *standardise* the columns, on the training data only.
- ▶ The intercept θ_0 should *not* be penalised: penalising it shrinks the fit towards zero rather than towards the data mean. Standard remedy: centre $\mathbf{X}$ and $\mathbf{y}$, fit without an intercept, recover $\hat{\theta}_0 = \bar{y} - \sum_{j \geq 1} \bar{x}_j \hat{\theta}_j$ afterwards – what `scikit-learn` does internally.

Tomorrow's Case 1

A second-order polynomial in one variable, fitted with OLS, Ridge and Lasso, scaled and unscaled, intercept penalised and not – so you see each of these statements happen in your own code. Below, standardised columns are assumed.

Why OLS is equivariant – and why Ridge and Lasso are not

A change of units multiplies each column by a constant: $X \rightarrow XD$, $D = \text{diag}(d_0, \dots, d_{p-1})$ (millimetres instead of metres: $d_j = 1000$). Three three-line computations:

$$\textbf{OLS: } \hat{\theta}(XD) = (DX^T XD)^{-1} DX^T y = D^{-1} \hat{\theta}(X) \Rightarrow XD D^{-1} \hat{\theta} = X \hat{\theta}.$$

Coefficients rescale by the inverse factors, every prediction is untouched: *equivariance* – the parametrisation moves, the model does not.

$$\textbf{Ridge: } DX^T XD + \lambda I = D (X^T X + \lambda D^{-2}) D \Rightarrow \hat{\theta}_{\text{Ridge}}(XD; \lambda) = D^{-1} (X^T X + \lambda D^{-2})^{-1} X^T y.$$

Rescaled Ridge = original-units Ridge with the sphere $\lambda \|\theta\|_2^2$ replaced by the *ellipsoid* $\lambda \theta^T D^{-2} \theta$: a different penalty, a different model, different predictions (unless $D \propto I$, which only relabels λ).

$$\textbf{Lasso: } \|y - XD\theta'\|_2^2 + \lambda \|\theta'\|_1 = \|y - X\theta\|_2^2 + \lambda \sum_j \frac{|\theta_j|}{d_j} \quad (\theta = D\theta').$$

Each feature pays its own price λ/d_j : the millimetre column pays a thousandth of the metre column. Which variables get selected becomes an artefact of the bookkeeping.

Units change the model: the numbers

Two features – a length in metres, a time in seconds – $n = 50$, $\lambda = 0.1$; then the length column converted to millimetres ($d_1 = 1000$) and everything refitted. From the chapter-3 program (full code in `week35tuesday.ipynb` and Section 3.13):

	OLS	Ridge	Lasso
max prediction change , m $\rightarrow$ mm	$8.9 \cdot 10^{-16}$	0.033	0.426
length coefficient, metres	1.982	1.916	1.123
length coefficient, mm ($\times 1000$)	1.982	1.982	1.984

- ▶ OLS: predictions identical to machine precision – equivariance, live.
- ▶ Ridge: predictions move; the fit on millimetre columns is verified to coincide (after undoing D) with the metre fit under the modified penalty $\lambda \theta^T D^{-2} \theta$ – the previous slide's algebra, to machine precision.
- ▶ Lasso: in metres the length coefficient is shrunk $1.98 \rightarrow 1.12$; in millimetres it escapes the penalty almost entirely (price $\lambda/1000$). Same data, same λ – different model.

Deriving the intercept formula, Eq. (3.90)

Single out θ_0 in the cost (no column of ones in X):

$$C(\theta_0, \dots, \theta_{p-1}) = \frac{1}{n} \sum_{i=0}^{n-1} \left(y_i - \theta_0 - \sum_{j=1}^{p-1} X_{ij} \theta_j \right)^2, \quad \frac{\partial C}{\partial \theta_0} = -\frac{2}{n} \sum_{i=0}^{n-1} \left(y_i - \theta_0 - \sum_{j=1}^{p-1} X_{ij} \theta_j \right) = 0.$$

The sum of the constant θ_0 is $n\theta_0$, so dividing by n :

$$\hat{\theta}_0 = \bar{y} - \sum_{j=1}^{p-1} \bar{x}_j \hat{\theta}_j$$

 (Eq. 3.90); one slope: $\theta_0 = \bar{y} - \theta_1 \bar{x}_1$ – the line passes through $(\bar{x}_1, \bar{y})$.

Substituting the optimal θ_0 back turns the cost into a *centred* problem – $\tilde{y} = y - \bar{y}1$, $\tilde{X}_{ij} = X_{ij} - \bar{x}_j$ – with

$$\hat{\theta} = \left(\tilde{X}^T \tilde{X} \right)^{-1} \tilde{X}^T \tilde{y} \quad (\text{OLS}), \quad \hat{\theta} = \left(\tilde{X}^T \tilde{X} + \lambda I \right)^{-1} \tilde{X}^T \tilde{y} \quad (\text{Ridge}).$$

Why this matters for the penalised methods

The intercept has *disappeared* from the optimisation: the penalty runs over $j = 1, \dots, p - 1$ only. Penalising θ_0 instead drags the whole fit towards zero and typically *raises* the MSE; the centred fit places the level of the data where the data put it. Tomorrow's Case 1 measures exactly this.

The Lasso: change one exponent

Replace the 2-norm penalty by the 1-norm,

$$\min_{\boldsymbol{\theta} \in \mathbb{R}^p} \left\{ \frac{1}{n} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2 + \lambda \|\boldsymbol{\theta}\|_1 \right\}, \quad \|\boldsymbol{\theta}\|_1 = \sum_{j=0}^{p-1} |\theta_j|.$$

The change looks slight; the consequences are not: the Lasso sets coefficients *exactly to zero* – it performs variable selection as part of the fit.

- ▶ No closed form: the stationarity condition $2\mathbf{X}^T\mathbf{X}\boldsymbol{\theta} + \lambda \text{sgn}(\boldsymbol{\theta}) = 2\mathbf{X}^T\mathbf{y}$ involves $\text{sgn}(\boldsymbol{\theta})$, undefined at 0 and dependent on the unknown – no matrix inversion solves it.
- ▶ But the problem is *convex* (quadratic + norm): a global minimum exists and is found reliably. What is lost is the formula, not the solution.

The cleanest case: soft thresholding

Take $X = I$ ($n = p$): the problem decouples into p scalar problems, one per coefficient. Solving each branch of the absolute value and checking consistency (Eqs. 3.60–3.63 of the book):

$$\underbrace{\hat{\theta}_i^{\text{OLS}} = y_i}_{\text{Eq. (3.60)}}$$

$$\underbrace{\hat{\theta}_i^{\text{Ridge}} = \frac{y_i}{1 + \lambda}}_{\text{Eq. (3.61)}}$$

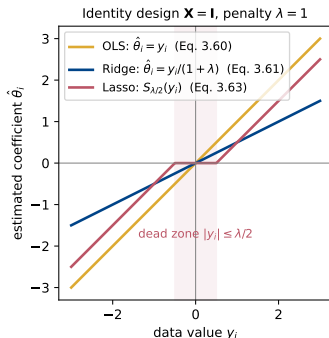
$$\underbrace{\hat{\theta}_i^{\text{Lasso}} = \begin{cases} y_i - \lambda/2 & y_i > \lambda/2, \\ y_i + \lambda/2 & y_i < -\lambda/2, \\ 0 & |y_i| \leq \lambda/2, \end{cases}}_{\text{Eq. (3.63)}}$$

the last being the *soft thresholding* operator $S_{\lambda/2}(y) = \text{sgn}(y) \max(|y| - \lambda/2, 0)$.

The comparison that contains everything

OLS leaves the coefficient alone. Ridge *multiplies* it by $1/(1 + \lambda)$ – zero only as $\lambda \rightarrow \infty$. The Lasso *subtracts* $\lambda/2$ from its magnitude and truncates at zero – every coefficient below the threshold is eliminated outright, at finite λ . Shrinkage versus selection, in one line.

Eqs. (3.60), (3.61) and (3.63), plotted



Each estimator as a function of the data value y_i it responds to ($\lambda = 1$):

- ▶ **OLS** is the identity: the coefficient reproduces the data, noise included.
- ▶ **Ridge** rotates the line down: every coefficient is multiplied by $1/(1 + \lambda)$ – large and small alike, and never to zero. *Proportional* shrinkage.
- ▶ **Lasso** translates the line towards zero and clips it: a *dead zone* $|y_i| \leq \lambda/2$ where the coefficient vanishes exactly, and a constant bias $\lambda/2$ on the survivors.

Note the large- $|y_i|$ behaviour: the Lasso subtracts a constant, not a fraction – it distorts strong signals *less* than Ridge while removing weak ones entirely. Code: `week35tuesday.ipynb`.

Pause and think 6: thresholds

Question

Orthonormal design, $\lambda = 1$. The OLS coefficients are $(3.0, 0.6, -0.4, -2.2)$. Compute the Ridge and Lasso coefficients. Which variables does each method keep?

Pause and think 6: thresholds

Question

Orthonormal design, $\lambda = 1$. The OLS coefficients are $(3.0, 0.6, -0.4, -2.2)$. Compute the Ridge and Lasso coefficients. Which variables does each method keep?

Answer

Ridge divides everything by $1 + \lambda = 2$: $(1.5, 0.3, -0.2, -1.1)$ – all four survive, all shrunk equally. Lasso subtracts $\lambda/2 = 0.5$ from each magnitude and truncates: $(2.5, 0.1, 0, -1.7)$ – the third variable is gone *exactly*, and at $\lambda \geq 1.2$ the second would follow. Ridge answers “how much of everything”; Lasso answers “which ones at all”.

Why the geometry does it

The constrained forms explain selection without calculus: minimise the squared error subject to

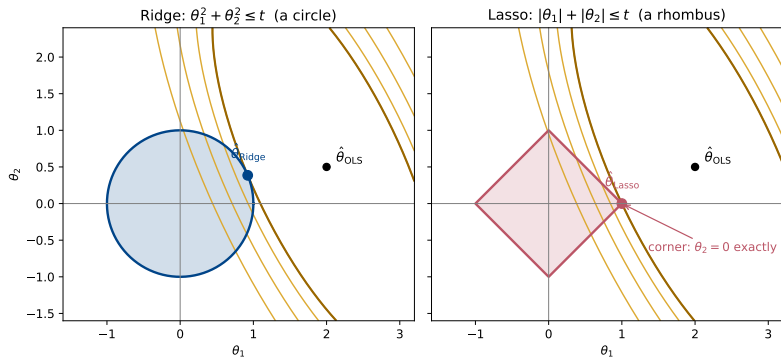
$$\|\boldsymbol{\theta}\|_2^2 \leq t \quad (\text{Ridge: a ball}), \quad \|\boldsymbol{\theta}\|_1 \leq t \quad (\text{Lasso: a diamond}).$$

The solution lies where the elliptical contours of the squared error first touch the constraint region.

- ▶ A ball has no distinguished points: contact happens at a generic location, all coordinates non-zero.
- ▶ A diamond has *corners*, and the corners lie on the axes, where coordinates vanish. Contours are far more likely to hit a protruding corner than a flat face – and every corner is a solution with zero coefficients.
- ▶ In p dimensions the 1-norm ball has corners, edges and faces of every dimension: each corresponds to a different set of eliminated variables.

The exact statement is the subdifferential/KKT condition two slides on; the picture – next slide, for two parameters – is the reason to believe it.

The geometry in two dimensions: circle vs. rhombus



Two parameters only: the Ridge ball is a *circle*, the Lasso diamond a *rhombus*. The ellipses are the contours of $\|y - X\theta\|_2^2$, centred on $\hat{\theta}_{OLS}$; the darker ellipse is the first one to touch each constraint region, and the touching point *is* the constrained solution (computed exactly here, same cost and same t in both panels). On the circle the contact is generic – both coordinates stay non-zero. On the rhombus the contour reaches the protruding *corner* first, and the corner sits on the axis: $\hat{\theta}_2 = 0$ exactly, the variable is eliminated. Code: `week35tuesday.ipynb`.

When is a coefficient zero? The exact condition

The absolute value is not differentiable at 0; the fix is the *subdifferential*, $\partial|\theta| = \{+1\}, \{-1\}$ or $[-1, 1]$ for $\theta > 0, < 0, = 0$. A convex function is minimal where 0 belongs to its subdifferential. With residual $r = y - X\hat{\theta}$ and column x_j :

$$\frac{2}{n}x_j^T r = \lambda \operatorname{sgn}(\hat{\theta}_j) \quad \text{if } \hat{\theta}_j \neq 0, \quad \left| \frac{2}{n}x_j^T r \right| \leq \lambda \quad \text{if } \hat{\theta}_j = 0.$$

Read the second condition again

A coefficient is zero because its column's correlation with the residual is *too small to pay the price* λ . A variable is excluded as long as it fails to earn its penalty.

Immediate corollary: the solution is $\hat{\theta} = 0$ exactly when

$$\lambda \geq \lambda_{\max} = \frac{2}{n} \max_j |x_j^T y|,$$

which is why every implementation computes a *path* of solutions on a grid running down from $\lambda_{\max}$.

Solving the Lasso: coordinate descent

Fix all coefficients but one; the scalar problem is soft thresholding on the partial residual

$$r^{(j)} = y - \sum_{k \neq j} x_k \theta_k:$$

$$\theta_j \leftarrow \frac{S_{\lambda/2}(x_j^T r^{(j)})}{x_j^T x_j}.$$

Cycle until nothing changes; convexity plus separability of the penalty guarantees convergence to the global minimum. This is what `sklearn.linear_model.Lasso` runs.

```
def soft_threshold(z, gamma):  
    return np.sign(z) * np.maximum(np.abs(z) - gamma, 0.0)  
  
def lasso_cd(X, y, lambda, n_iter=1000, tol=1e-8):  
    n, p = X.shape; theta = np.zeros(p)  
    col_norms = np.sum(X**2, axis=0); r = y - X @ theta  
    for _ in range(n_iter):  
        theta_old = theta.copy()  
        for j in range(p):  
            r += X[:, j] * theta[j] # add column j back  
            theta[j] = soft_threshold(X[:, j] @ r, lambda * n / 2.0) / col_norms[j]  
            r -= X[:, j] * theta[j] # remove the update  
        if np.max(np.abs(theta - theta_old)) < tol: break  
    return theta
```

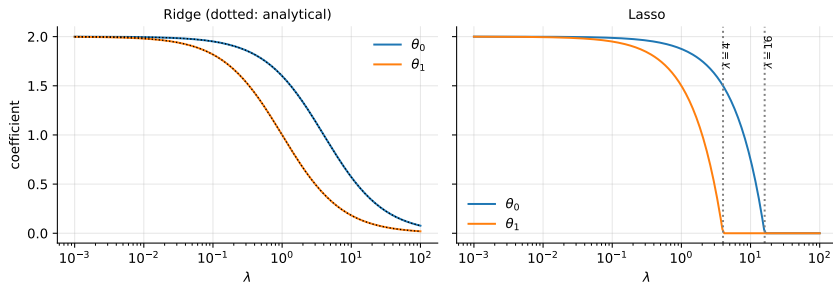
An example (Section 3.10): the three estimators on one problem

$$y = \begin{bmatrix} 4 \\ 2 \\ 3 \end{bmatrix}, \quad X = \begin{bmatrix} 2 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix} : \quad \text{three observations, two features; the third row only adds error.}$$

$$\hat{\theta}_{\text{OLS}} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}, \quad \hat{\theta}_{\text{Ridge}} = \begin{bmatrix} \frac{8}{4 + \lambda} \\ \frac{2}{1 + \lambda} \end{bmatrix}, \quad \hat{\theta}_{\text{Lasso}} = \begin{bmatrix} \max((16 - \lambda)/8, 0) \\ \max(2 - \lambda/2, 0) \end{bmatrix}.$$

- ▶ Ridge shrinks the two coefficients by *different* factors, $4/(4 + \lambda)$ and $1/(1 + \lambda)$: the better-determined feature ($\sigma_0^2 = 4$ vs $\sigma_1^2 = 1$) resists shrinkage more. Neither ever reaches zero.
- ▶ The Lasso eliminates θ_1 at $\lambda = 4$ and θ_0 at $\lambda = 16$ – finite thresholds, exact zeros.

Coefficient paths: shrinkage vs. selection at a glance



The Ridge curves approach the axis asymptotically; the Lasso curves reach it and stop. The dotted lines are the analytical results of the previous slide – reproduced by `scikit-learn` to machine precision, *once the conventions are matched*.

Pause and think 7: the convention trap

Question

scikit-learn minimises $\|y - X\theta\|_2^2 + \alpha\|\theta\|_2^2$ for Ridge, but $\|y - X\theta\|_2^2/(2n) + \alpha\|\theta\|_1$ for Lasso. Our course convention puts $1/n$ in front of the squared error for both. What α must you pass to each to reproduce our λ ? Why should you *always* check this before comparing your own code with a library?

Pause and think 7: the convention trap

Question

scikit-learn minimises $\|y - X\theta\|_2^2 + \alpha\|\theta\|_2^2$ for Ridge, but $\|y - X\theta\|_2^2/(2n) + \alpha\|\theta\|_1$ for Lasso. Our course convention puts $1/n$ in front of the squared error for both. What α must you pass to each to reproduce our λ ? Why should you *always* check this before comparing your own code with a library?

Answer

Ridge: multiply our objective by n – the penalty becomes $n\lambda\|\theta\|_2^2$, so $\alpha = n\lambda$ (for the toy problem of the previous slide the book uses the no- $1/n$ convention, and there $\alpha = \lambda$ directly).
Lasso: multiply our objective by $1/2$ – $\alpha = \lambda/2$ (or $\lambda/(2n)$ from the no- $1/n$ convention). The same library uses *different* conventions for the two methods, and every textbook differs from every implementation. Mismatched factors change every number while leaving the method unchanged – reconcile them explicitly, never by assumption. This is exercise material for tomorrow.

Pause and think 8: when the Lasso is the wrong tool

Question

You have $p = 10\,000$ gene-expression features, $n = 200$ patients, and you expect a handful of genes to matter – but genes come in strongly correlated clusters that switch on together. What will the Lasso do with such a cluster, why is that a problem for the science, and which modification repairs it?

Pause and think 8: when the Lasso is the wrong tool

Question

You have $p = 10\,000$ gene-expression features, $n = 200$ patients, and you expect a handful of genes to matter – but genes come in strongly correlated clusters that switch on together. What will the Lasso do with such a cluster, why is that a problem for the science, and which modification repairs it?

Answer

The Lasso picks *one* member of each correlated cluster, essentially arbitrarily, and zeroes the rest; under resampling a different member wins, so the selected gene list is unstable and biologically misleading. (It can also select at most $n = 200$ variables in total.) The *elastic net*, penalising $\alpha\|\boldsymbol{\theta}\|_1 + (1 - \alpha)\|\boldsymbol{\theta}\|_2^2$, combines selection with Ridge's grouping behaviour and tends to keep or drop correlated clusters together. Remember also the week-34 lesson: predictions can be fine even when the attribution is unstable.

Summary, and what comes next

- ▶ Exercise 3: training MSE falls monotonically with complexity, test MSE turns up – MSE, R^2 and MAE (Section 3.5) mean something only on held-out data.
- ▶ Gauss–Markov: OLS is the best *linear unbiased* estimator – a restricted competition; $\text{MSE} = \text{bias}^2 + \text{variance}$ opens the door to biased estimators.
- ▶ Ridge: through the SVD, Eqs. (3.46)–(3.47), the only change is $1/\sigma_i \rightarrow \sigma_i/(\sigma_i^2 + \lambda)$ – each singular direction shrunk by the factor of Eq. (3.48); variance uniformly smaller; Hoerl–Kennard guarantees some $\lambda > 0$ beats OLS.
- ▶ Lasso: Eqs. (3.60)/(3.61)/(3.63) in one picture – Ridge rotates the line, the Lasso translates and clips it; exact zeros, variable selection; convex but no closed form; coordinate descent.
- ▶ Section 3.10's example: Ridge shrinks the two coefficients by different factors, never to zero; the Lasso kills them at $\lambda = 4$ and 16.
- ▶ Ball vs diamond – the geometry behind selection. Conventions ($1/n$, $1/2n$, α vs λ) differ between every book and library: check, don't assume. Bayesian reading of the penalties: week 37.

This week and next

Tomorrow: the exercise-3 figure revisited, then Section 3.13 hands-on – scaling and the intercept in OLS, Ridge and Lasso on a second-order polynomial. Next week: resampling, cross-validation, bias–variance in full.